Letter to the Editor

May 19, 1992

From: Jack W. Reeves, San Jose, CA

To: Livleen Singh, Editor, *C++ Journal*, Port Washington, NY

Dear Editor,

Thank you for printing my letter of August 27, 1991 commenting on software design. I agree (in principal, if not in detail) with most of your reply. What we all want is to find ways to produce better software and help our industry "mature into a disciplined science." My problem is, about ten years ago I came to the conclusion that, as an industry, we do not understand what a software design really is. I am even more convinced of this today. I do not claim that my point of view is correct, but I have found it very useful in explaining why some things work and why others do not. There is a very subtle, but very important point which is being missed. This is the difference between doing software design and what a finished software design really is. I would like to elaborate upon this point.

Allow me to begin with the final part of your reply. I made the statement (supplying context) "It may not be a very good (software) design, but a (software) design it is." You suggest comparing software design with bridge design and created the following statement "It may not be a very good bridge, but a bridge it is." Here the word "bridge" is substituted for "(software) design." The interpretation seems to be "Would you trust something that was built with little or no design?" The obvious answer is "Of course not!" The comparison is not valid, however. The fact that it seems valid to most of the industry is exactly the point I am trying to make.

Instead of changing the sentence, change the context instead. Now the statement would read "It may not be a very good (bridge) design, but a (bridge) design it is." Would you volunteer to be the first across this bridge? The immediate answer might be "No, a bad design is no better than no design!" A little thought will show that an equally valid answer is "What bridge?" Until you actually build the bridge from the design there is no need to worry about crossing it. The point is that we don't build bridges from scratch designs. Before the bridge actually gets built, the design will be refined considerably. We will do analysis. We will build computer models and run simulations. We may even build scale models and test them in wind tunnels and other ways. We will do everything we can to make sure the design is a good design before we build the bridge. We call this "engineering" (and sometimes, despite everything, it still isn't completely right...there was this bridge in Tacoma).

Back to software. We in the software industry also refine our designs, only we don't get to call it engineering. We call it "testing and debugging". This phase of the software lifecycle takes a long time. All too often it takes longer than planned. Unfortunately, it is often not enough and the final designs that we turn into deliverable software are still not as good as they should be. This seems like a fact of software life. Many people lament it and ask why we software developers do not "engineer" our designs better? Many explanations are offered, but never the one most obvious to me -- simple economics. Software is dirt cheap to build.

Am I crazy? I don't think so! Compiling and linking 50,000 lines of C++ code on your 486 may seem to take forever, but how would you like to assemble a circuit card with 50,000 discreet components, or build a bridge with 50,000 structural elements? We don't construct mathematical proofs of software correctness or run our code through symbolic executors because it takes less time and effort to just build it and test it. We probably would get better software if we did more of the former, but we don't. Why not?

There are probably lots of reasons, but I would like to suggest that many of them derive from our failure to consider testing and debugging as part of the software design process. We would like for it to go away completely. Since it will not, we try to treat it as some sort of "quality assurance" function and spend as little time, effort, and money on it as we can get away with. We consider it a shame of the software industry that testing and debugging take up half the typical software development lifecycle. Is it really so bad? I suspect that most engineers in other disciplines haven't a clue about what percentage of their time is spent actually creating a design and what is spent on testing and debugging the result. Some industries are probably better than software. I am pretty sure that other industries are actually much worse. Consider what it must take to "design" a new airliner.

I get somewhat testy when people start making gratuitous comparisons between software design and other engineering disciplines. Major microprocessors have been shipped with bugs in their logic, bridges have collapsed, dams broken, airliners fallen out of the sky, and thousands of automobiles and other consumer products recalled - all within recent memory and all the result of design errors.

The problem with software is - design is not just important, it is basically everything. Saying that programmers should not have to design is like saying fish should not have to swim. When I am coding, I am designing. I am creating a software design out of the void. Sometimes the algorithm is simple and the design is trivial. Often times, I have to design data structures and the algorithms to match. There may be alternatives and I have to choose between them based upon my perception of the advantages and disadvantages of each. Sometimes I decide that the design is getting too complex and I specify one or more

sub-modules. When I have finished the design, I test it to see how good it is and refine it to make it better. Refinements not only come from finding errors in the original design, but from other sources such as peer walkthroughs and formal reviews. The bottom line is that my design must be correct, or every piece of software built from it will be erroneous. Therefore I concentrate on doing it right, and it takes mental effort and skill, just like any other creative design activity.

Nevertheless, most software systems are quite large and quite complex and my one module is only a small part. While I am concentrating on the details of code design for module X, there may be hundreds of other modules and thousands of other details that I can not possibly worry about at the same time. There are also important aspects of software design which do not fall cleanly into the categories of data structures and algorithms. It is these "other aspects" that most people mean when they say software design.

It is true that programmers do not want to worry about the "high level aspects" of a design when they are designing code. They often end up having to worry about them anyway. The high level design clearly affects the detailed design. The converse is also true. The details of internal design may (or *should*) help decide amongst high level alternatives. Refining all aspects of the design is a process that should be happening throughout the development cycle. If some aspect of the design is "frozen" out of the refinement process, you can potentially end up with a poor or even unworkable final design.

Some of my colleagues have interpreted my harangues on this subject as "Jack says forget design and just start coding". Nothing could be further from the truth (though I see how they get that impression). I am not against traditional software design. We desperately need good design at all levels. It doesn't matter whether we call the early process top level design, structural design, module design, or whatever. What I have been arguing for are two changes in perception. First, that we recognize that the results of the early design steps are not a complete software design any more than the first rough sketches are a complete bridge design. Second, that we capture our design thinking using a notation that is a true skeleton software design. That means using a programming language.

Ultimately, the computer doesn't care how we get to a final software design any more than the steeplejack building a bridge cares how the bridge design was refined and validated. All that matters to either of them is that the design they are working from be sufficient to allow them to correctly build the product. On the other hand, what it takes to create a good software design obviously matters a lot to those of us responsible for creating one. The better the early design, the less work needed refining it later. That is what we are really talking about, is it not.

What I am arguing for is not less software design, but a realistic software design process. We need to recognize the difference between *designing software* and *a software design*. It makes sense to use any tools and techniques that we find help us during the design process. It does not make sense to forget what is the real software design. When we have worked out some aspect of software design, we should not let incorrect comparisons with hardware engineering disciplines keep us from correctly documenting our work in a software design. *YES, WE SHOULD CODE IT*. If what we are really doing is software design, then everything we do will somehow be reflected in code. We might as well write the code (or that portion of it that makes sense) when we make the decisions that affect that code.

I know all the arguments for "language independent" software design notations. They all ignore a fundamental problem. Software design involves translating concepts from some problem space into a programming language. This translation has to be done by human beings, and since our programming languages are usually totally inadequate to express the concepts of the problem space directly, it is usually a difficult and error prone process. When we translate concepts from one form to another, especially complex ones, we often loose important information. If several translations are involved, we are likely to end up with a final product that lost too much vital information, that does not accurately reflect our original concept, and/or that simply contains errors. This is compounded several times when the people actually doing the translation are different for each step. Remember, there is nothing sacred about C++ (or Ada, or C, or Smalltalk, or LISP, or any programming language). It is not the native language of our computers. Fundamentally, programming languages are just a design notation themselves. I do not see any point in introducing extra translation steps if they can be avoided.

There are a couple of problems with my "code as design" approach, which I acknowledge. The first is that even the best programming languages have serious weaknesses as tools for expressing certain aspects of a software design. The information is in the code (if it isn't, then it wasn't software design information) but it is very difficult to get it out in human readable form. These are the "other aspects" of a software design mentioned above. The second problem is similar. There is going to be information from the problem space that went into the software design, but that can not be reconstructed from the software design itself. We want to capture this information in case we need to change the software design later. The typical source code comment is not an adequate mechanism.

Both of these problems mean that auxiliary documentation is as important to software design as it is to any other engineering discipline, if not more so. We must recognize auxiliary documentation as such, however, and not confuse it with the software design.

What we really need is more expressive programming languages. This is what led to my statement about C++ being a major advance in software design art. C++ is a more expressive programming language, which makes it a better software design tool.

As a final topic, consider what my point of view reveals about traditional software development processes. Ultimately, all software design processes end up validating and refining the final design via a build/test cycle. Any pretense otherwise is just silliness. Yet, traditional MIL-STD and other waterfall model development processes will not even allow writing one line of code until a certain tonnage of auxiliary documentation has been produced and reviewed. Often, the people who produce this documentation then go on to other things leaving a group of new, and usually much younger and less experienced people to actually generate the real software design. It is hardly surprising (to me anyway) that this process has fallen into such disrepute that no real developers advocate it. What are they trying instead?

Now we have rapid prototyping and spiral development. In my view, it is easy to see why these are replacing the waterfall method. Both of these are just excuses for writing code earlier in the development cycle so that the process of refining the design via build/test can begin sooner. They also typically get the same people involved in both the top level design and the actual code design. Not surprisingly, these two approaches are both seen as significant improvements. Even the best of the traditional approaches continue to try to break software design into disjoint steps with separate notations and products, and then they continue to wonder why they have problems getting a final software product that is correct.

Projects done in Ada have shown significant improvements in the time necessary to integrate, test and debug (at the expense of some extra top level design effort). Structured programming was considered a breakthrough in its time. Object oriented design and C++ are taking the world by storm. Forget all the explanations offered for these phenomenon, and consider what hasn't worked. CASE tools? Popular, yes; universal, no. Structure charts? Same thing. Likewise, Warner-Orr diagrams, Booch diagrams, object diagrams, you name it. Each has its strengths, and a single fundamental weakness - it really isn't a software design. Ultimately, improvements in programming techniques are overwhelmingly more important to software development than anything else.

There seems to be a collective fantasy of the software community that if we could just find the right graphical design notation so that software designs would look like other engineering designs, then we could take our place as software engineers alongside the other disciplines. I disagree. Engineering is about how you do the process, not about whether the final product needs a CAD system to render it. We in the software business are so close, but so far away. Software is so -- well, *soft*. A software system can represent

anything. This, plus the economics of the build/test cycle, makes it very unlikely that we will find general purpose methods for validating a software design other than the current trial and error. We can improve the process, however. Maybe if we started treating software development as a homogeneous design process, and concentrated on improving the most important phases (programming, debug and test), we might find our industry to be more of a disciplined science than we think it is.

I still do not know if I have made my point, but in summary:

- Real software is what runs on computers. This means that real software is not C++ (or any other programming language).
- Real software is built by computers (via compilers and linkers).
- This means that a source code listing (in any programming language) is really a software design.
- It follows from the above that real software is incredibly cheap to build, and getting cheaper all the time as computers get faster.
- Software is incredibly expensive to design. This is true in both an absolute sense (because of its ever increasing complexity), and relative to the cost of a software build.
- Programming is a design activity - a good software design process recognizes this and doesn't hesitate to code when coding makes sense.
- Coding actually makes sense a whole lot more often than traditional software design processes would have you believe.
- Testing and debugging are design activities - they are the software equivalent of the analysis, simulation, modeling, and testing phases of other engineering disciplines. The goal is to validate and improve the design before the final product is built. A good software design process recognizes this and doesn't try to disown or short change the steps.
- There are other design activities - call them top level design, module design, or whatever. A good software design process recognizes this and deliberately includes the steps.
- All design activities interact. A good software design process recognizes this and allows the design to change, sometimes radically, as other design steps reveal the need.

- Many different software design notations are potentially useful - as auxiliary documentation (it would be nice to have some tools that help us generate and maintain auxiliary documentation automatically from the actual source code. This would be particularly useful in maintaining a graphical representation of the structural aspects of the design).
- Ultimately, real advances in software development depend upon advances in programming. C++ is a good step in the right direction. For software engineering's sake, we need a great many more steps.

###

Historical Note

In an email correspondence discussing the publication of this letter and its relationship to the original letter,

> "The sequence of events was as follows: I read an article about software design in one issue of C++ Journal. I sent a letter to the editor complaining about something (I don't remember what, and can not find the letter on my system these days). The editor printed the letter, and a response. What I have attached is the letter I emailed to the editor as a response to his response. I think you will find it rather familiar. I personally find it better written than the article itself. The editor Livleen Singh offered to allow me to turn my points into a full article, which I did."

About the Author

Jack W. Reeves is a senior software developer with over 30 years experience in the industry. He has worked on systems ranging from simulators for the space shuttle, military command and control systems, air traffic control systems, medical imaging systems, financial data distribution systems, embedded systems, drivers, and utilities. He has exclusively been an OO developer for the last 15 years.